Operation Analytics and Investigating Metric Spike

Project Report Case Study

By Gayatri Palkar

CASE STUDY 1: JOB DATA ANALYSIS

Note: As the data was insufficient, I have added few unique as well as duplicate records, for better data analysis

Tasks and Insights:

1. Jobs Reviewed Over Time:

- **Objective:**

Calculate the number of jobs reviewed per hour for each day in November 2020.

- **Your Task:**

Write an SQL query to calculate the number of jobs reviewed per hour for each day in November 2020.

- **Steps to get to solution:**

1. Filter the data to include only records from November
2. Extract the hour from the timestamp - use hour() function for this
3. Group the data by date and hour.
4. Count the number of jobs reviewed for each group and display them.

- **Query:**

```
#task1
SELECT DATE(ds) AS date, HOUR(ds) AS hour, COUNT(*) AS jobs_reviewed
FROM job_data
WHERE ds BETWEEN '2020-11-01' AND '2020-11-30'
GROUP BY DATE(ds), HOUR(ds)
ORDER BY date, hour;
```

- **Output:**

Result Grid | Filter Rows:

date	hour	jobs_reviewed
2020-11-01	1	1
2020-11-01	2	1
2020-11-01	3	1
2020-11-02	1	1
2020-11-02	2	1
2020-11-02	3	1
2020-11-03	1	1
2020-11-03	2	1
2020-11-03	3	1
2020-11-04	1	2
2020-11-05	1	1
2020-11-06	1	1
2020-11-07	1	1
2020-11-08	1	1
2020-11-09	1	1
2020-11-10	1	1
2020-11-11	1	1
2020-11-12	1	1
2020-11-13	1	1
2020-11-14	1	1
2020-11-15	1	1

Result 10

- **Insights:**

1. **This analysis helps to identify the hours of the day when the most jobs are reviewed.**
2. **For all hours the average number of jobs reviewed is 1, with a very rare exception of 2 per hour**
3. **Here we can say that for a normal person/actor , the number of jobs reviewed per hour is 1 on average.**

2. Throughput Analysis:

- **Objective:**

Calculate the 7-day rolling average of throughput (number of events per second).

- **Your Task:**

Write an SQL query to calculate the 7-day rolling average of throughput. Additionally, explain whether you prefer using the daily metric or the 7-day rolling average for throughput, and why.

- **Steps to get to solution:**

1. **Calculate the total number of events happening in each day.**
2. **Compute the 7 day rolling average of events per day.**

- **Query:**

```sql
#task2
WITH daily_events AS (
    SELECT DATE(ds) AS date, COUNT(*) AS events
    FROM job_data
    GROUP BY DATE(ds)
)
SELECT date, events,
       AVG(events) OVER (ORDER BY date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS 7day_avg_throughput
FROM daily_events;
```

- **Output:**

Result Grid | Filter Rows: | Export:

date	events	7day_avg_throughput
NULL	8	8.0000
2020-11-01	3	5.5000
2020-11-02	3	4.6667
2020-11-03	3	4.2500
2020-11-04	2	3.8000
2020-11-05	1	3.3333
2020-11-06	1	3.0000
2020-11-07	1	2.0000
2020-11-08	1	1.7143
2020-11-09	1	1.4286
2020-11-10	1	1.1429
2020-11-11	1	1.0000
2020-11-12	1	1.0000
2020-11-13	1	1.0000
2020-11-14	1	1.0000
2020-11-15	1	1.0000
2020-11-16	1	1.0000
2020-11-17	1	1.0000
2020-11-18	1	1.0000
2020-11-19	1	1.0000
2020-11-20	1	1.0000
2020-11-21	1	1.0000
2020-11-22	1	1.0000
2020-11-23	3	1.2857
2020-11-24	2	1.4286

- **Insights:**

1. **The 7-day rolling average smooths out daily fluctuations and provides a clearer view of trends over time.**
2. **Helps us track the performance of the review process over a week. This helps in identifying any gradual improvements or declines in throughput.**

3. **Decides whether to adjust processes or resources based on throughput trends. For example, if throughput is consistently low, it may indicate a need for more reviewers or process improvements.**

Preference Between Daily Metric and 7-Day Rolling Average:

```
#task2_2
SELECT DATE(ds) AS date, COUNT(*) AS daily_events
FROM job_data
GROUP BY DATE(ds)
ORDER BY date;
```

Result Grid | Filter Rows: | Export:

date	daily_events	7_day_avg_throughput
NULL	8	8.00
2020-11-01	3	5.50
2020-11-02	3	4.67
2020-11-03	3	4.25
2020-11-04	2	3.80
2020-11-05	1	3.33
2020-11-06	1	3.00
2020-11-07	1	2.00
2020-11-08	1	1.71
2020-11-09	1	1.43
2020-11-10	1	1.14
2020-11-11	1	1.00
2020-11-12	1	1.00
2020-11-13	1	1.00
2020-11-14	1	1.00
2020-11-15	1	1.00
2020-11-16	1	1.00
2020-11-17	1	1.00
2020-11-18	1	1.00
2020-11-19	1	1.00
2020-11-20	1	1.00
2020-11-21	1	1.00
2020-11-22	1	1.00
2020-11-23	3	1.29

- **Daily Metric - Provides precise daily performance but can be noisy due to daily fluctuations but gives the exact number of events processed each day. Thus helps in short term decision making.**
- **7-Day Rolling Average - Smooths out short-term fluctuations, revealing longer-term trends for better strategic decisions. Better for checking performance over time. So good for long term decision making**
- **Here the Data is for a very short term (one month at max), not many months or years are considered. Hence I feel that Daily Metric is much more useful.**

3. Language Share Analysis:

- **Objective:**

Calculate the percentage share of each language in the last 30 days.

- **Your Task:**

Write an SQL query to calculate the percentage share of each language over the last 30 days.

- **Steps to get to solution:**

1. Filter data for the last 30 days.
2. Count the number of occurrences for each language.
3. Calculate the percentage share of each language.

- **Query:**

```
#task3
SELECT language,
       COUNT(*) * 100.0 / SUM(COUNT(*)) OVER () AS percent_share
FROM job_data
WHERE ds >= DATE_SUB((SELECT MAX(ds) FROM job_data), INTERVAL 30 DAY)
GROUP BY language
ORDER BY percent_share DESC;
```

- **Output:**

```
#task3
SELECT language,
       ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (),2) AS percent_share
FROM job_data
WHERE ds >= DATE_SUB((SELECT MAX(ds) FROM job_data), INTERVAL 30 DAY)
GROUP BY language
ORDER BY percent_share DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

language	percent_share
English	16.67
Spanish	13.89
French	13.89
German	11.11
Chinese	11.11
Italian	8.33
Russian	8.33
Japanese	8.33
Korean	8.33

- **Insights:**

1. **Understands the distribution of different languages in the reviewed content. This can help in identifying the primary languages that require focus.**
2. **English followed by Spanish and French are the top 3 languages in which content is being reviewed being 16%, 14%and 14% respectively**
3. **German and Chinese are also considered 11.11%**

4. Duplicate Rows Detection:

- **Objective:**

Identify duplicate rows in the data.

- **Your Task:**

Write an SQL query to display duplicate rows from the job_data table.

- **Steps to get to solution:**

1. **Group data by all columns.**
2. **Identify groups with more than one occurrence.**

- **Query:**

```sql
#task4
SELECT job_id, actor_id, event, language, time_spent, org, ds, COUNT(*)
FROM job_data
GROUP BY job_id, actor_id, event, language, time_spent, org, ds
HAVING COUNT(*) > 1;
```

- **Output:**

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

job_id	actor_id	event	language	time_spent	org	ds	COUNT(*)
131	2031	skip	English	30	A	2020-11-23 01:00:00	2
132	2032	decision	Spanish	25	B	2020-11-24 02:00:00	2
133	2033	transfer	French	20	C	2020-11-25 03:00:00	2

- **Insights:**

1. **Identifies and addresses data quality issues.**
2. **Duplicate rows can lead to inaccurate analysis and insights.**
3. **Ensures the uniqueness of records.**

PROJECT REPORT CASE 1

1. Project Description

The dataset used in this analysis is focused on job reviews.

2. Approach

My approach to handle this project was as follows:

1. **Went through the dataset provided, see what columns exist and what they mean in terms of the complete analysis. Import Dataset into MYSQL WB.**
2. **Added more records to the dataset (as instructed) as many of the sql queries were not giving any satisfactory output, so I looked at the data as a whole and added more records, also added few duplicate records as there were none**
3. **Solved the Tasks one by one, took help from videos wherever required**
4. **Thought over why we are finding the query and searched if the query had any significance with data**
5. **Made a report of insight found by me.**

3. Tech-Stack Used

For Executing this project I have used MySQL Workbench: Version 8.0.36 Community Version. It has helped me access and add to or make modifications to data as required. It also helped me run my SQL queries for fetching data and making analysis on it

4. Insights

The dataset tracks job reviews with details like review date, job ID, reviewer ID, review action (decision, skip, transfer), content language, review time in seconds, and reviewer organization. It helps analyze review timings, reviewer performance, language distribution, and data quality for operational efficiency. Gained insight that English, Spanish, French are top 3 languages to be reviewed.

5. Result

The overall objective of this analysis was to understand and improve the operational efficiency of a company.

We determined peak review times and ensured if staffing is adequate.

To track the number of events processed we identified if any trends existed over time.

Understood the distribution of different languages in the content reviewed.

Checked for duplicate records to maintain data integrity and uniqueness.

CASE STUDY 2: INVESTIGATING METRIC SPIKE

Tasks and Insights:

1. **Weekly User Engagement:**
   - **Objective:**

Measure the activeness of users on a weekly basis.

- **Your Task:**

Write an SQL query to calculate the weekly user engagement.

- **Steps to get to solution:**

1. **Extract the year and week from the 'occured_at' column.**
2. **Group the results by year, week, and 'user_id'.**
3. **Count the number of events per user per week.**

- **Query:**

```
#qUERIES----------------------------------
#TASK 1
SELECT YEAR(occured_at) AS year, WEEK(occured_at) AS week, user_id, COUNT(*) AS engagement_count
FROM events
GROUP BY YEAR(occured_at), WEEK(occured_at), user_id
ORDER BY year, week;
```

- **Output:**

Result Grid | Filter Rows:

year	week	user_id	engagement_count
2014	17	8	2
2014	17	22	8
2014	17	66	14
2014	17	163	22
2014	17	172	15
2014	17	209	9
2014	17	232	39
2014	17	239	9
2014	17	253	18
2014	17	287	16
2014	17	319	10
2014	17	365	7
2014	17	373	8
2014	17	404	3
2014	17	434	5
2014	17	441	28
2014	17	492	19
2014	17	589	11
2014	17	591	13
2014	17	660	5
2014	17	688	8
2014	17	714	7
2014	17	729	50
2014	17	745	25
2014	17	746	5
2014	17	822	4

- **Insights:**

1. **This query calculates the number of events (actions) each user performs on a weekly basis.**
2. **The 'engagement_count' metric helps determine which users are most active during any given week.**
3. **Trends can be analyzed over time to identify peaks in engagement, potentially correlating them with specific events, promotions, or changes in the product.**

2. User Growth Analysis:

- **Objective:**

Analyze the growth of users over time for a product.

- **Your Task:**

Write an SQL query to calculate the user growth for the product.

- **Steps to get to solution:**

1. **Extract the year and month from the 'created_at' column.**
2. **Group the results by year and month.**
3. **Count the number of new users per month.**

- **Query:**

```sql
#TASK 2
SELECT YEAR(created_at) AS year, MONTH(created_at) AS month, COUNT(*) AS new_users
FROM users
GROUP BY YEAR(created_at), MONTH(created_at)
ORDER BY year, month;
```

- **Output:**

Result Grid | Filter Rows:

year	month	new_users
2013	1	160
2013	2	160
2013	3	150
2013	4	181
2013	5	214
2013	6	213
2013	7	284
2013	8	316
2013	9	330
2013	10	390
2013	11	399
2013	12	486
2014	1	552
2014	2	525
2014	3	615
2014	4	726
2014	5	779
2014	6	873

- **Insights:**

1. **This query calculates the number of new users registering each month.**
2. **By tracking user growth over time, one can identify trends, assess the effectiveness of marketing campaigns, and detect any significant changes in user acquisition.**
3. **Analyzing the results can help to understand seasonal trends or the impact of external factors on user growth.**
4. **We can see here that Users increase over a period of time, in each month the product is reaching new users.**

3. **Weekly Retention Analysis:**

- **Objective:**

Analyze the retention of users on a weekly basis after signing up for a product.

- **Your Task:**

Write an SQL query to calculate the weekly retention of users based on their sign-up cohort.

- **Steps to get to solution:**

1. **Identify the sign-up week for each user.**
2. **Identify the weeks when users were engaged.**

3. Join these results to calculate retention based on sign-up cohorts.

- **Query:**

```sql
#TASK 3
WITH signup_weeks AS (
    SELECT user_id, YEAR(created_at) AS signup_year, WEEK(created_at) AS signup_week
    FROM users),
engagement_weeks AS (
    SELECT user_id, YEAR(occured_at) AS engagement_year, WEEK(occured_at) AS engagement_week
    FROM events)
SELECT s.signup_year, s.signup_week, e.engagement_year, e.engagement_week,
    COUNT(DISTINCT e.user_id) AS retained_users
FROM signup_weeks s
JOIN engagement_weeks e ON s.user_id = e.user_id
WHERE
    (e.engagement_year > s.signup_year OR
    (e.engagement_year = s.signup_year AND e.engagement_week >= s.signup_week))
GROUP BY s.signup_year, s.signup_week, e.engagement_year, e.engagement_week
ORDER BY retained_users desc;
```

- **Output:**

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch

signup_year	signup_week	engagement_year	engagement_week	retained_users
2014	33	2014	33	261
2014	34	2014	34	259
2014	32	2014	32	245
2014	30	2014	30	238
2014	24	2014	24	229
2014	27	2014	27	222
2014	29	2014	29	221
2014	28	2014	28	215
2014	25	2014	25	207
2014	26	2014	26	201
2014	22	2014	22	196
2014	23	2014	23	196
2014	31	2014	31	193
2014	33	2014	34	187
2014	19	2014	19	185
2014	21	2014	21	183
2014	20	2014	20	176
2014	32	2014	33	174
2014	30	2014	31	171
2014	25	2014	26	165
2014	18	2014	18	163
2014	27	2014	28	161
2014	28	2014	29	161

Average retained users weekwise

Result Grid | Filter Rows: | Export:

signup_year	signup_week	average_retained_users
2014	33	152.0000
2014	34	151.0000
2014	32	127.0000
2014	30	102.3333
2014	31	90.2000
2014	27	87.1250
2014	28	85.1429
2014	29	84.0000
2014	26	70.6667
2014	25	67.1000
2014	24	63.6364
2014	23	58.9167
2014	22	53.3846
2014	21	46.0000
2014	20	45.4667
2014	19	42.3125
2014	17	34.2632
2014	18	34.1667

- **Insights:**

1. **This query calculates the number of users retained on a weekly basis after their sign-up week.**
2. **Retention rates are crucial for understanding user loyalty and the long-term value of users.**
3. **The results help identify patterns in user retention and areas where improvements may be needed to increase retention rates.**
4. **As the week goes on, the number of users retained is more**

4. **Weekly Engagement Per Device:**

- **Objective:**

Measure the activeness of users on a weekly basis per device.

- **Your Task:**

Write an SQL query to calculate the weekly engagement per device.

- **Steps to get to solution:**

1. **Extract the year and week from the 'occured_at' column.**
2. **Group the results by year, week, and device.**
3. **Count the number of events per device per week.**

- **Query:**

```
#TASK 4
SELECT YEAR(occured_at) AS year, WEEK(occured_at) AS week, device,
    COUNT(*) AS engagement_count
FROM events
GROUP BY YEAR(occured_at), WEEK(occured_at), device
ORDER BY engagement_count desc;
```

- **Output:**

Result Grid | Filter Rows: | Export:

year	week	device	engagement_count
2014	31	macbook pro	3649
2014	30	macbook pro	3603
2014	27	macbook pro	3593
2014	28	macbook pro	3496
2014	32	macbook pro	3372
2014	26	macbook pro	3341
2014	18	macbook pro	3334
2014	33	macbook pro	3243
2014	19	macbook pro	3189
2014	29	macbook pro	3189
2014	34	macbook pro	3182
2014	23	macbook pro	3149
2014	20	macbook pro	3129
2014	22	macbook pro	3073
2014	21	macbook pro	3069
2014	24	macbook pro	3067
2014	25	macbook pro	2966
2014	30	lenovo think...	2614

- **Insights:**

1. **This query calculates the number of events (actions) performed on each device every week.**

2. The results highlight which devices are most used by users, which can inform decisions about platform-specific optimizations.
3. We can see that most of the users own a Macbook Pro and are accessing the content/email using that. So most of the emails/ads are to be addressed to Macbook pro owners.

5. **Email Engagement Analysis:**

- **Objective:**

Analyze how users are engaging with the email service.

- **Your Task:**

Write an SQL query to calculate the email engagement metrics.

- **Steps to get to solution:**

1. Extract the year and month from the occured_at column.
2. Group the results by year, month, and action.
3. Count the number of each action per month.

- **Query:**

```sql
#TASK 5
SELECT YEAR(occured_at) AS year, MONTH(occured_at) AS month, action,
    COUNT(*) AS action_count
FROM email_events
GROUP BY YEAR(occured_at), MONTH(occured_at), action
ORDER BY action_count desc;
```

- **Output:**

Result Grid | Filter Rows: | Export

year	month	action	action_count
2014	8	sent_weekly_digest	16480
2014	7	sent_weekly_digest	15902
2014	6	sent_weekly_digest	13155
2014	5	sent_weekly_digest	11730
2014	8	email_open	5978
2014	7	email_open	5611
2014	6	email_open	4658
2014	5	email_open	4212
2014	7	email_clickthrough	2721
2014	6	email_clickthrough	2274
2014	5	email_clickthrough	2023
2014	8	email_clickthrough	1992
2014	8	sent_reengageme...	1073
2014	7	sent_reengageme...	933
2014	6	sent_reengageme...	889
2014	5	sent_reengageme...	758

- **Insights:**

1. **Here we can see that certain email content gets more user engagement. Weekly digests get more clicks out of other emails and reengagement.**
2. **The results can help to understand user interactions with email communications, such as the number of opens, clicks, bounces, etc.**
3. **Analyzing these metrics provides insights into the effectiveness of email campaigns and user engagement with email content.**

PROJECT REPORT CASE 2

6. Project Description

This project focuses on analyzing user data to understand their engagement, growth, retention, and interaction with email services.

7. Approach

My approach to handle this project was as follows:

1. **Went through the dataset provided, imported Tables into MYSQL WB.**
2. **Changed datetime column of data (as instructed)**
3. **Solved the Tasks one by one, took help from videos wherever required**
4. **Thought over why we are finding the query and searched if the query had any significance with data**
5. **Made a report of insight found by me..**

8. Tech-Stack Used

For Executing this project I have used MySQL Workbench: Version 8.0.36 Community Version. It has helped me access and add to or make modifications to data as required. It also helped me run my SQL queries for fetching data and making analysis on it

9. Insights

The key observations we learn from the project are that it helps determine which users are most active during any given week, one can identify trends, patterns in user retention and areas of improvements. The results can help to understand user interactions with email communications, such as the number of opens, clicks, bounces, etc.

10. Result

- **Weekly User Engagement: Users are most active on specific days and occasions.**
- **User Growth: More users join during marketing campaigns.**
- **Retention: Early engagement leads to better long-term retention.**
- **Device Preference: Users mostly use Macbook Pro.**
- **Email Interaction: Weekly digests get more user engagement.**
- **These insights help improve user experience, boost retention, and optimize marketing efforts.**

Image source: Google Images